

Data Mining

Lab - 7 (Part-2)

¥@\$# Kakadiya | 23010101121 | 30/7/2025

Step 1: Load the Dataset

Load the `Tdata.csv` file and display the first few rows.

```
In [3]: import pandas as pd
```

```
In [4]: df = pd.read_csv('../data/Tdata.csv')
df
```

```
Out[4]:
```

	Transaction	bread	butter	coffee	eggs	jam	milk
0	T1	1	1	0	0	0	1
1	T2	1	1	0	0	1	0
2	T3	1	0	0	1	0	1
3	T4	1	1	0	0	0	1
4	T5	1	0	1	0	0	0
5	T6	0	0	1	1	1	0

Step 2: Drop the 'Transaction' Column

We're only interested in the items (not the transaction IDs).

```
In [5]: df_items = df.drop(columns = ['Transaction'])
df_items
```

Out[5]:

	bread	butter	coffee	eggs	jam	milk
0	1	1	0	0	0	1
1	1	1	0	0	1	0
2	1	0	0	1	0	1
3	1	1	0	0	0	1
4	1	0	1	0	0	0
5	0	0	1	1	1	0

Step 3: Count Single Items

See how many transactions include each item.

```
In [6]: df_items.sum()
```

Out[6]:

bread	5
butter	3
coffee	2
eggs	2
jam	2
milk	3

dtype: int64

Step 4: Define Apriori Function

This function finds frequent itemsets of size 1, 2, and 3 with minimum support.

```
In [7]: from itertools import combinations

def find_frequent_itemsets(df, min_support):
    n = len(df)
    result = []

    for k in [1, 2, 3]: #for 1-item, 2-item, 3-item sets
        for items in combinations(df.columns, k):
            mask = df[list(items)].all(axis = 1)
            support = mask.sum() / n

            if support >= min_support:
                result.append((frozenset(items), round(support, 2)))

    return result
```

Step 5: Run Apriori

Set `min_support = 0.6` and display the frequent itemsets.

```
In [8]: frequent_itemsets = find_frequent_itemsets(df_items, 0.5)

for itemset, support in frequent_itemsets:
    print(f'{set(itemset)} → support: {support}')

{'bread'} → support: 0.83
{'butter'} → support: 0.5
{'milk'} → support: 0.5
{'butter', 'bread'} → support: 0.5
{'milk', 'bread'} → support: 0.5
```

Step 6 Display as a DataFrame

```
In [9]: result_df = pd.DataFrame(frequent_itemsets, columns = ['Itemset', 'Support'])
result_df
```

```
Out[9]:
```

	Itemset	Support
0	(bread)	0.83
1	(butter)	0.50
2	(milk)	0.50
3	(butter, bread)	0.50
4	(milk, bread)	0.50

Orange Tool : - >Generate Same Frequent Patterns in Orange tools

```
In [ ]:
```

Extra : - > Define Apriori Function without itertools

```
In [10]: def find_frequent_itemsets(df, min_support):
n = len(df)
result = []
columns = list(df.columns)

    for i in range(len(columns)):
        items = [columns[i]]
        mask = df[items].all(axis=1)
        support = mask.sum() / n
        if support >= min_support:
            result.append((frozenset(items), round(support, 2)))

    for i in range(len(columns)):
        for j in range(i + 1, len(columns)):
            items = [columns[i], columns[j]]
            mask = df[items].all(axis=1)
            support = mask.sum() / n
            if support >= min_support:
                result.append((frozenset(items), round(support, 2)))

    for i in range(len(columns)):
        for j in range(i + 1, len(columns)):
            for k in range(j + 1, len(columns)):
                items = [columns[i], columns[j], columns[k]]
                mask = df[items].all(axis=1)
                support = mask.sum() / n
                if support >= min_support:
                    result.append((frozenset(items), round(support, 2)))

    return result
```

```
In [11]: frequent_itemsets = find_frequent_itemsets(df_items, 0.5)

result_df = pd.DataFrame(frequent_itemsets, columns = ['Itemset', 'Support'])
result_df
```

Out[11]:

	Itemset	Support
0	(bread)	0.83
1	(butter)	0.50
2	(milk)	0.50
3	(butter, bread)	0.50
4	(milk, bread)	0.50